

# Day2 学习文档 v2.0 : Agent Runtime —— 谁才是真正的大脑？

本文是《从零实现 Agent Runtime》学习阶段的 Day2 正式学习文档。

Day2 的核心目标不是学习某个 Agent 框架的 API，而是站在 Runtime 设计者的角度，理解一个 Agent 到底由谁控制、如何循环、如何维护状态，以及 LLM、Tool、Conversation、Context、Summary 之间的职责边界。

## 与上一章的联系

Day1 主要解决的是宏观认知问题：

- Agent 不是简单的 Chat。
- Agent 也不等于 Tool Calling。
- Agent 至少由 LLM、Tool、Runtime、Context、Memory 等部分共同组成。
- 学习 Agent 最好的路径，不是直接使用框架，而是先自己实现一个最小 Runtime。

Day2 在 Day1 的基础上继续向下挖一层，核心问题从“Agent 是什么”变成：

Agent 运行时，到底是谁在负责控制流程？

这一章最终形成了一个关键判断：

LLM 负责推理，Runtime 负责管理。真正让 Agent 工作起来的，不是模型本身，而是 Runtime。

## 目录

Agent 不是 Tool，而是 Runtime

Chat 为什么不是 Agent？

真正循环的是 Runtime，不是 LLM

LLM 与 Runtime 的职责划分

Conversation 与 Context 的区别

ContextBuilder 为什么存在？

Conversation 永远不会 Summary

Summary 是 Runtime 主动触发的

Summary Cache 的作用

Tool 的设计

Runtime 的职责最终确认

Agent 的结束是谁决定的？

今天我们推导出的 Runtime 数据模型

Day2 最终推导出的 mini-agent-runtime

今天最大的认知升级

写书 TODO

写书素材

## 1. Agent 不是 Tool，而是 Runtime

很多初学者会把 Agent 理解为：

Agent = LLM + Tool

这个理解不算完全错误，但它只看到了表层。

更准确地说，真正的 Agent 应该是：

Agent = Runtime + LLM + Tool

其中：

- LLM 负责 Reason，也就是根据输入进行推理和生成下一步响应。
- Tool 负责 Act，也就是执行真实世界中的动作，例如查天气、读文件、调用接口、执行命令。
- Runtime 负责 Manage，也就是控制整个 Agent 的生命周期。

如果没有 Runtime，LLM 即使能返回一个 Tool Call，也只是输出了一段结构化文本。它不会自己执行函数，不会自己追加上下文，不会自己判断是否需要进入下一轮，也不会自己处理错误、超时、审批和状态更新。

所以，真正让 Agent 成为 Agent 的，不是工具本身，而是 Runtime 对整个循环的调度能力。

## 2. Chat 为什么不是 Agent？

普通 Chat 的流程非常简单：

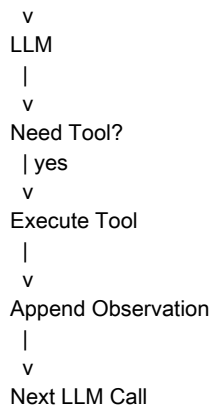
```
User
|
v
LLM
|
v
Answer
```

它的特点是：

- 一次请求。
- 一次回复。
- 没有工具执行。
- 没有中间状态。
- 没有循环。
- 没有 Runtime 级别的控制逻辑。

而 Agent 的流程不是一次性结束的。Agent 会在 LLM 和 Runtime 之间反复循环：

```
User
|
v
Runtime
|
v
Build Context
|
```



这就是 ReAct 思想：

Reason -> Act -> Observe -> Reason -> Act -> Observe

Chat 只是一次推理。

Agent 是由 Runtime 驱动的多轮推理与执行循环。

因此，Chat 和 Agent 的本质区别不在于有没有调用模型，而在于有没有 Runtime Loop。

### 3. 真正循环的是 Runtime，不是 LLM

Day2 中一个非常重要的误区是：

Agent 是不是 LLM 一次又一次地调用自己？

答案是否定的。

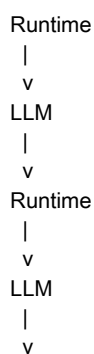
LLM 本质上只是一个函数：

```
const response = await llm.chat(context)
```

它接收输入，然后返回输出。它不会：

- 调用自己；
- 执行 Tool；
- 更新 Conversation；
- 写数据库；
- 生成 Summary Cache；
- 控制循环；
- 判断是否强制停止。

真正发生的是：



Runtime

也就是说，每一次 LLM 调用都是 Runtime 主动发起的。

Runtime 调用 LLM，LLM 返回响应，Runtime 解析响应。如果响应中包含 Tool Call，Runtime 执行工具，把结果追加到 Conversation，再重新构建 Context，然后再次调用 LLM。

所以，“LLM -> LLM -> LLM”这个说法只是表象。

更准确的描述是：

Runtime -> LLM -> Runtime -> LLM -> Runtime -> LLM

Agent 的自主性不是 LLM 自己递归出来的，而是 Runtime 通过循环、状态更新和工具执行模拟出来的。

## 4. LLM 与 Runtime 的职责划分

Day2 最大的收获之一，是把 LLM 与 Runtime 的职责边界彻底拆开。

LLM 只负责一件事：

根据 Runtime 提供的 Context，生成下一步 Response。

这个 Response 可能是：

- 一段最终回答；
- 一个 Tool Call；
- 多个 Tool Calls；
- 一个中间推理结果；
- 一个结构化计划。

但是，LLM 不负责执行。

真正执行动作的是 Runtime。

可以把二者关系理解为：

LLM = Decision  
Runtime = Execution + State Management + Control

例如，LLM 返回：

```
{
  "tool": "weather",
  "args": {
    "city": "Shanghai"
  }
}
```

这并不表示 LLM 调用了天气工具。它只是“建议”或“请求”调用这个工具。

真正执行的是 Runtime：

```
const tool = toolRegistry.get("weather")
const result = await tool.execute({ city: "Shanghai" })
```

职责边界可以总结为：

LLM 决定下一步想做什么。  
Runtime 决定是否允许、如何执行、如何记录、是否继续。

这也是后续 Human Approval、权限控制、超时、重试、审计日志等能力能够成立的原因。

## 5. Conversation 与 Context 的区别

Day2 中另一个非常重要的概念是 Conversation 与 Context 的区别。

很多人会把它们混为一谈，但在 Agent Runtime 中，它们是两个不同层级的对象。

### Conversation

Conversation 是完整的真实历史，是 Source of Truth。

它记录的是整个会话中实际发生过的所有消息和事件，例如：

```
User
Assistant
Tool
Assistant
User
Assistant
```

Conversation 的特点：

- 保存完整历史；
- 通常会落库；
- 是审计、恢复、调试的依据；
- 原则上不应该随意修改；
- 不等于每次发给 LLM 的输入。

### Context

Context 是 Runtime 每一次调用 LLM 前临时构建出来的输入。

它可能包含：

```
System Prompt
+
Summary
+
Memory
+
最近 N 条 Conversation Message
+
当前用户消息
```

也就是说，LLM 看到的从来不是完整 Conversation，而是 Runtime 根据策略构建出的 Context。

关系如下：

```
Conversation
|
v
ContextBuilder
|
v
Context
|
v
LLM
```

一句话总结：

Conversation 是事实；Context 是视图。

## 6. ContextBuilder 为什么存在？

ContextBuilder 的存在，是因为 Conversation 和 LLM 所需输入不是同一个东西。

如果 Conversation 很短，Runtime 可以简单地把消息原样发给 LLM：

```
const context = conversation.messages
```

但真实 Agent 很快会遇到问题：

- Conversation 越来越长；
- Token 成本越来越高；
- LLM Context Window 有上限；
- 某些历史需要总结；
- 某些记忆需要插入；
- 某些工具结果需要保留；
- 某些旧消息需要裁剪。

因此，Runtime 需要一个独立的 ContextBuilder。

ContextBuilder 的职责是：

```
读取 Conversation
|
v
选择保留哪些消息
|
v
读取 Summary Cache
|
v
读取 Memory
|
v
插入 System Prompt
|
v
生成最终 Context
```

ContextBuilder 本身通常是普通业务逻辑，而不是 LLM 自动完成的事情。

它可以很简单：

```
function buildContext(conversation) {
  return [
    systemPrompt,
    ...conversation.messages.slice(-20),
  ]
}
```

也可以很复杂：

```
System Prompt
+
Project Memory
+
Summary Cache
+
Recent Tool Observations
+
Recent Messages
+
```

Current User Message

但无论复杂还是简单，决策权都在 Runtime，而不在 LLM。

## 7. Conversation 永远不会 Summary

Day2 讨论中，一个关键误区是：

当 Conversation 很长时，是不是把前 100 条总结成一条 Summary，然后 push 回 Conversation，再删除原来的 100 条？

这个方案看起来直觉上合理，但在 Runtime 设计中并不推荐。

原因是 Conversation 是真实历史。

如果把 Summary 写回 Conversation 并替代原始消息，会带来很多问题：

- 原始历史丢失，无法审计；
- Debug 时无法还原现场；
- Summary 本身可能有误，无法纠正；
- 下一次 Summary 时边界变得混乱；
- 无法区分“真实发生的消息”和“压缩后的缓存”。

正确设计是：

```
Conversation:
1
2
3
...
200

Summary Cache:
summary of 1~180

Context:
Summary(1~180)
+
181~200
```

Conversation 不变。

Summary 只是 ContextBuilder 在构建 Context 时使用的辅助状态。

因此：

```
Conversation = Fact
Summary      = Cache
Context      = View
```

这是 Day2 最重要的架构结论之一。

## 8. Summary 是 Runtime 主动触发的

另一个重要问题是：

Summary 是 LLM 自己决定做的吗？

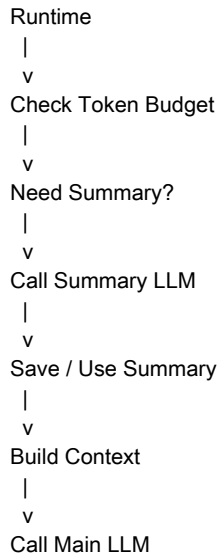
答案也是否定的。

Summary 是 Runtime 的策略，不是 LLM 的自发行为。

Runtime 会根据某些条件决定是否触发 Summary，例如：

- 当前 Context 超过 token 限制；
- Conversation 消息数超过阈值；
- 最近一次 Summary 已经覆盖太少；
- 进入新阶段，需要压缩旧阶段内容；
- 成本控制需要减少重复输入。

流程大致是：



这里要注意：

用于 Summary 的 LLM 和用于正式推理的 LLM，在职责上不同。

Summary LLM 只是 Runtime 调用的一个能力，类似一个内部工具。

所以可以说：

LLM 在 Summary 场景里不是 Agent，而是 Runtime 的一个组件。

这进一步说明，Agent 的主导权始终在 Runtime。

## 9. Summary Cache 的作用

如果每次构建 Context 都重新 Summary，成本会非常高。

例如 Conversation 已经有 1000 条消息，如果每一轮都重新总结前 900 条，就会造成：

- Token 成本高；
- 延迟高；
- 结果不稳定；
- 重复计算；
- Summary 内容可能每次略有不同。

所以生产级 Runtime 通常会引入 Summary Cache。



Summary Cache 可以理解为一个 Checkpoint :

```
summary: "已经完成了需求分析、项目初始化和第一个 Runtime Loop 设计..."
coveredUntil: 180
```

其中 coveredUntil 表示这个 Summary 覆盖到了哪一条 Conversation Message。

之后如果 Conversation 增长到 250 条，ContextBuilder 可以构建：

```
Summary(1~180)
+
Message 181~250
```

这样就不需要每次重新总结 1~180。

Summary Cache 一般应该独立存储，而不是混进 Conversation。

可以有类似的数据结构：

```
interface ConversationSummary {
  conversationId: string
  summary: string
  coveredUntilMessageId: string
  updatedAt: Date
}
```

这也解释了为什么企业级 Agent Runtime 的存储通常会拆成：

```
Conversation Store
Summary Store
Memory Store
```

它们职责不同，不应该混在一起。

## 10. Tool 的设计

Day2 对 Tool 的设计形成了一个比较明确的结论：

```
const tool = {
  name,
  description,
  parameters,
  execute,
}
```

这种对象结构非常自然，因为它同时满足 LLM 和 Runtime 的需求。

LLM 需要：

- name：知道工具叫什么；
- description：知道什么时候应该使用；
- parameters：知道参数结构。

Runtime 需要：

- execute：真正执行工具逻辑。

也就是说，Tool 有两面：

```
给 LLM 看的一面：name / description / parameters
给 Runtime 用的一面：execute
```

LLM 不会执行 execute。

LLM 只会根据工具描述决定是否请求调用某个工具。

真正调用：

```
await tool.execute(args)
```

的是 Runtime。

这个职责边界非常重要，因为后面 Human Approval、Tool Permission、Tool Timeout、Tool Retry 都会建立在这个结构上。

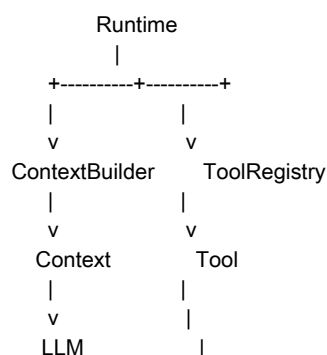
## 11. Runtime 的职责最终确认

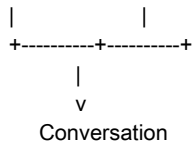
经过 Day2 的讨论，我们最终确认 Runtime 是整个 Agent 的 Orchestrator。

Runtime 至少负责：

- 接收用户输入；
- 写入 Conversation；
- 构建 Context；
- 调用 LLM；
- 解析 LLM Response；
- 判断是否存在 Tool Call；
- 查找 Tool；
- 进行权限检查；
- 执行 Tool；
- 追加 Tool Result；
- 控制 Runtime Loop；
- 触发 Summary；
- 维护 Summary Cache；
- 处理错误；
- 控制最大步数；
- 控制超时；
- 处理 Human Approval；
- 判断是否结束；
- 输出最终结果给用户。

如果用一张图表示：





LLM 很重要，但 LLM 不是整个系统的控制中心。

Runtime 才拥有最终控制权。

## 12. Agent 的结束是谁决定的？

Day2 对“谁决定结束”也做了拆分。

直觉上看，好像是 LLM 决定结束。

例如 LLM 返回：

```
{
  "content": "任务已完成，这是最终答案。"
}
```

没有 Tool Call，Runtime 就可以结束。

所以可以说：

LLM 提供结束建议。

但真正最终决定是否结束的，仍然是 Runtime。

Runtime 可以因为以下原因结束：

- LLM 返回 Final Answer；
- 达到最大循环次数；
- 达到超时时间；
- 用户点击 Stop；
- Human Approval 被拒绝；
- Tool 执行失败且不可恢复；
- Token Budget 不允许继续；
- Runtime 策略认为继续执行风险过高。

因此，更准确的说法是：

```
LLM suggests finish.
Runtime decides finish.
```

也就是：

LLM 建议结束，Runtime 最终裁决。

这个原则非常重要，因为任何生产级 Agent 都不能把最终控制权完全交给 LLM。

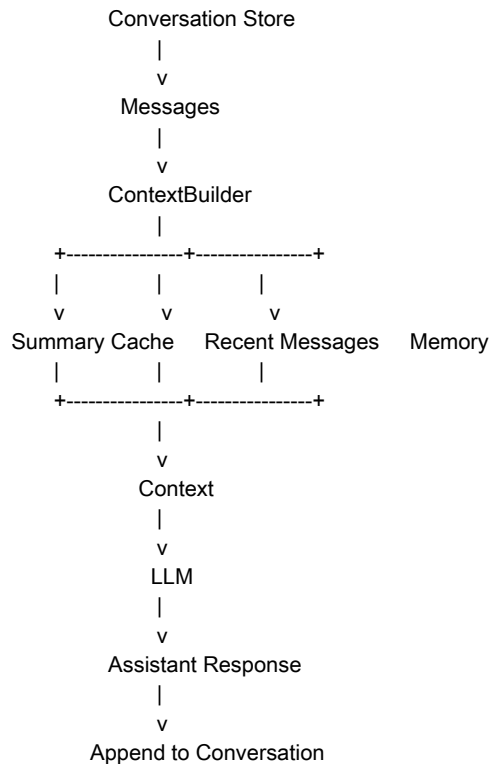
## 13. 今天我们推导出的 Runtime 数据模型

Day2 中，我们初步推导出了一个真实 Agent Runtime 可能需要的数据模型。

核心对象包括：

Conversation  
Message  
Summary Cache  
Tool  
Context  
Runtime State

它们之间的关系如下：



在这个模型中：

- Conversation 是事实来源；
- Message 是事实记录；
- Summary Cache 是压缩缓存；
- Context 是每轮 LLM 调用的输入；
- ContextBuilder 是转换器；
- Runtime State 记录当前执行状态；
- ToolRegistry 管理可调用工具。

这个模型虽然还很粗糙，但已经具备一个 Agent Runtime 的核心骨架。

## 14. Day2 最终推导出的 mini-agent-runtime

Day2 还没有正式写业务代码，但我们已经推导出了 mini-agent-runtime 的第一版核心结构。

最小 Runtime 可以长这样：

```
class Runtime {
private conversation
private tools
private llm
private contextBuilder
private maxSteps
```

```

async run(input: string) {
  this.conversation.append({
    role: "user",
    content: input,
  })

  for (let step = 0; step < this.maxSteps; step++) {
    const context = this.contextBuilder.build(this.conversation)
    const response = await this.llm.chat(context)

    this.conversation.append(response.message)

    if (!response.toolCalls?.length) {
      return response.message
    }

    for (const toolCall of response.toolCalls) {
      const tool = this.tools.get(toolCall.name)
      const result = await tool.execute(toolCall.args)

      this.conversation.append({
        role: "tool",
        toolCallId: toolCall.id,
        content: result,
      })
    }
  }

  throw new Error("Runtime stopped because maxSteps was reached.")
}

```

这段伪代码里已经包含了 Agent Runtime 最核心的机制：

```

Append User Message
|
v
Build Context
|
v
Call LLM
|
v
Append Assistant Message
|
v
Need Tool?
|
v
Execute Tool
|
v
Append Tool Result
|
v
Next Loop

```

它还没有 Memory、Streaming、Human Approval、MCP、Workflow，但它已经是一个真正的 Agent Runtime 雏形。

## 15. 今天最大的认知升级

Day2 最大的变化，是我们从“使用 Agent”的视角，切换到了“设计 Agent Runtime”的视角。

学习前的直觉可能是：

Agent = LLM + Tool

学习后的理解是：

Agent = Runtime + LLM + Tool

学习前可能觉得：

LLM 在控制一切。

学习后应该知道：

LLM 只是根据 Context 生成下一步 Response。  
Runtime 才负责控制流程、管理状态、执行工具和决定结束。

学习前可能觉得：

Conversation 就是发给 LLM 的上下文。

学习后应该知道：

Conversation 是完整事实。  
Context 是 Runtime 构建出的本次输入。  
ContextBuilder 是二者之间的转换器。

学习前可能觉得：

Summary 是把历史消息合并后写回去。

学习后应该知道：

Summary 是 Cache，不是 Fact。  
Conversation 不应该被 Summary 覆盖。

Day2 的一句话总结是：

写 Chat 应用，是在调用一个模型；写 Agent Framework，是在设计一个 Runtime。

## 16. 写书 TODO

未来把 Day2 改写成正式 Chapter 时，需要补充以下问题。这里只列问题，不写答案。

- Runtime 的历史背景：为什么 Agent 发展到一定阶段一定会出现 Runtime？
- Chat API、Tool Calling、Agent Runtime 三者的演进关系是什么？
- ReAct 与 Runtime Loop 的关系应该如何系统说明？
- Runtime Loop 是否可以用状态机建模？
- Runtime 与 Orchestrator、Scheduler、State Machine 的关系是什么？
- OpenAI Agents SDK 中哪个部分对应 Runtime？
- Mastra 中哪个部分对应 Runtime？
- LangGraph 中哪个部分对应 Runtime？
- Claude Code / Cursor 这类 Coding Agent 的 Runtime 可能如何设计？
- Conversation、Context、Memory、Summary 四个概念需要更正式的定义。
- ContextBuilder 应该画出完整流程图。
- Summary Cache 的生产级实现需要补充数据库设计。
- coveredUntil / checkpoint 机制需要补充示例。

- Token Window 管理策略需要单独补充，包括 token 估算、缓存、裁剪、分层上下文。
- Runtime 为什么必须拥有最终控制权，需要增加更多安全案例。
- Human Approval 在 Runtime 中的位置需要通过流程图说明。
- Tool 的 Schema 设计需要对照 OpenAI / Anthropic / Mastra 的真实格式。
- Runtime 的错误处理、重试、超时、取消机制需要补充。
- mini-agent-runtime 第一版代码需要在后续章节真正实现。
- 需要补充 Sequence Diagram：一次完整 Tool Call 的执行链路。
- 需要补充 Class Diagram：Runtime、Conversation、ContextBuilder、ToolRegistry、LLMClient。
- 需要补充 Interview 问答，例如“Conversation 和 Context 有什么区别？”。
- 需要补充一节“为什么不能让 LLM 直接控制 Tool 执行？”
- 需要补充一节“为什么 Summary 不应该写回 Conversation？”
- 需要补充一节“Runtime 为什么是 Agent 的大脑，而不是 LLM？”

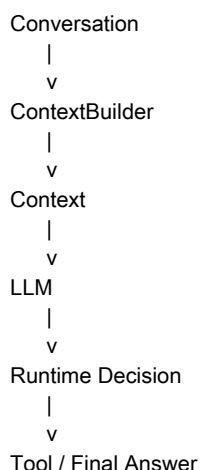
## 17. 写书素材

以下内容可以在未来写成正式 Chapter 时复用、扩展或改写。

### 核心句子

- Agent 不是 Tool，而是 Runtime 驱动的系统。
- LLM 负责推理，Runtime 负责管理。
- Tool Call 不是 Tool Execution。
- LLM 决定想调用什么工具，Runtime 决定是否执行以及如何执行。
- Conversation 是事实，Context 是视图。
- ContextBuilder 是把 Conversation 转换成 LLM 输入的策略层。
- Summary 是 Cache，不是 Fact。
- LLM 建议结束，Runtime 最终裁决。
- 写 Chat 应用，是在调用一个模型；写 Agent Framework，是在设计一个 Runtime。

### 可复用架构图



Runtime -> LLM -> Runtime -> Tool -> Runtime -> LLM

Conversation = Source of Truth

Summary = Cache

Context = View

## 可复用代码骨架

```
while (true) {
  const context = contextBuilder.build(conversation)
  const response = await llm.chat(context)

  conversation.append(response.message)

  if (!response.toolCalls?.length) {
    return response.message
  }

  for (const toolCall of response.toolCalls) {
    const result = await toolRegistry.execute(toolCall)
    conversation.append({
      role: "tool",
      toolCallId: toolCall.id,
      content: result,
    })
  }
}
```

## 可复用类名

- Runtime
- Conversation
- ConversationMessage
- Context
- ContextBuilder
- Tool
- ToolRegistry
- LLMClient
- SummaryCache
- RuntimeState

## 可复用比喻

- Runtime 像 Agent 的大脑中的调度系统。
- LLM 像推理器，只负责根据输入生成下一步。
- Conversation 像完整审计日志。
- Context 像每次递给模型的临时材料包。
- Summary Cache 像历史记录的可压缩索引。
- ContextBuilder 像把数据库事实转换成模型输入的渲染器。

## 未来章节衔接

Day2 已经确定 Runtime 是 Agent 的核心。下一步应该进入 Runtime 内部设计：



- Runtime 需要哪些类？
- Runtime 需要维护哪些状态？
- ToolRegistry 如何设计？
- LLMClient 如何抽象？
- ContextBuilder 的第一版实现应该长什么样？

这些问题将自然引出 Day3：Runtime Architecture。